# Review and Optimization of the zzcollab analysis-Archetype Dockerfile Template: A Technical White Paper

*2026-06-28 10:32 PDT*

**Status: Final (v1).** This paper documents a completed review and optimization of the Dockerfile that zzcollab generates for analysis-archetype research compendia. The optimization is implemented and verified: two generator changes (F-8 and F-9, Section 6 change log) cut the cold build 61 percent, confirmed across all three profiles. Section 5 records what was done, what was tried and rejected, and the open items deferred to future work. Baseline commit `e9e60a3` (`main`); the changes are on branch `feat/dockerfile-build-optimization` (PR #28).

This paper assumes fluency in R and data analysis but not in container tooling. Terms of art (Docker, BuildKit, PPM, renv internals, and so on) are collected with plain definitions in Appendix C; Section 9 gives a line-by-line reading of the generated Dockerfile.


## Abstract

zzcollab does not ship a static Dockerfile. It generates one per project from a shell function, `generate_dockerfile_inline` in `modules/docker.sh`, parameterised by the project's R version, base image, research archetype, and dependency-declaration mode. This paper reviews the Dockerfile produced for an analysis-archetype project that uses the `minimal` Docker profile (`rocker/r-ver`), using a concrete generated artifact as the specimen (Appendix A). We document the generation pipeline, catalogue the defects and assumptions identified, and act on the two with the largest build-time cost. The review began inspection-based; the findings were then driven to verification by repeated instrumented builds, and two generator changes (parallelising the language-server install, and excluding renv from its own restore) cut the cold build by 61 percent. Each finding states plainly how it was confirmed and how serious it is, and the change log (Section 6) records what was built, measured, and in two cases tried and reverted.


## 1. Scope and the artifact under review

### 1.1 Terminology: profile versus archetype

zzcollab overloads the word 'profile'. The term denotes a Docker base-image bundle (`minimal`, `tidyverse`, `rstudio`), and it formerly included a profile named `analysis`, which was renamed to `tidyverse` in commit `dbebb09` precisely because it collided with the `analysis` research archetype. A research archetype (`analysis`, and others) selects the directory scaffold and project intent, independent of the Docker base.

The specimen reviewed here is an analysis-*archetype* project configured with the `minimal` Docker *profile*, that is, a `rocker/r-ver` base rather than `rocker/tidyverse`. Where this paper says 'the analysis-archetype Dockerfile', it means the Dockerfile generated for that combination. The generator logic is largely shared across profiles; findings that are profile-independent are marked as such.

### 1.2 The specimen

The artifact is the `Dockerfile` of a scaffolded test project, `~/prj/msc/peng1`, generated 2026-06-28 from template version 0.1.0. Its salient parameters, taken from the project's `.zzcollab-state` and `renv.lock`:

- R version 4.6.0; base `rocker/r-ver:4.6.0`, digest-pinned.
- Install mode `renv`; `renv.lock` declares only `renv` 1.1.5 and `tinytest` 1.4.3.
- PPM snapshot `noble/2026-06-28` (regenerated; an earlier generation used the 2026-06-27 snapshot, which does not affect any finding).

The verbatim artifact is reproduced in Appendix A. The relevant generator excerpt is in Appendix B.

## 2. The generation pipeline

The Dockerfile is emitted by `generate_dockerfile_inline` (`modules/docker.sh`) via a single here-document. The function resolves four derived inputs before emission:

- **Ubuntu codename**, from the `get_ubuntu_codename` helper, a `case` over the R minor version: 4.2 and 4.3 map to `jammy`; 4.4 and 4.5 to `noble`; every other version, including 4.6, falls through the default arm to `noble`.
- **PPM URL**, the dated Posit Package Manager snapshot for that codename, used to pin binary packages.
- **FROM spec**, `${base_image}:${r_version}@${image_digest}` when a digest was resolved, otherwise the tag alone.
- **Install block**, branched at generation time on the presence of `renv.lock`: `renv` mode restores the lockfile; otherwise `description` mode installs declared dependencies with `pak`. A true build-time branch is not possible because Docker cannot `COPY` an optionally-present file.

The choice of install mode is recorded as both a build `ARG` and an OCI `LABEL`, so the image is self-describing. This is sound provenance practice, as is the digest-pinned base, the dated PPM snapshot wired in three places (the `ENV` override, `Rprofile.site`, and the lockfile repositories), and the `RENV_LOCK_HASH` build argument that ties the restore layer's cache key to the lockfile content. These elements are not in dispute and should be preserved by any optimization.

## 3. Findings

Each finding states, in plain terms, how it was confirmed and how serious it is. Where a finding was confirmed only by reading the generator and the generated file, the text says so; where it was instead checked by an actual build, a runtime test, or a query to the package registry, that is stated too. Findings already acted on are noted as fixed.

### F-1. Pre-FROM build args: one inert, one a Makefile metadata channel (fixed)

The generator emitted, before `FROM`, the args `BASE_IMAGE`, `R_VERSION`, and `USERNAME`. The `FROM` line is a generation-time literal (`${base_image}:${r_version}@${image_digest}`) and references none of them, so at first reading all three look inert with respect to the docker build. That was the original framing of this finding, and it was partly wrong.

Acting on it surfaced the correction. `R_VERSION` is genuinely unread: the `FROM` ignores it, no `--build-arg R_VERSION` is passed, and nothing else consults it. It was removed. The duplicate pre-`FROM` `USERNAME` was also removed (the post-`FROM` `ARG USERNAME` is the one `useradd`/`USER` use; see F-4). But removing `BASE_IMAGE` broke `make r`: the project `Makefile` parses it back out of the generated Dockerfile by text (`grep '^ARG BASE_IMAGE=' Dockerfile`) to derive the profile label shown when entering the container, and a second grep recovers `USERNAME`. So `ARG BASE_IMAGE` is not inert at all; it is a metadata side-channel that the build ignores but the tooling reads. The regression was caught before the change was pushed.

Resolution (applied): keep `ARG BASE_IMAGE` (with a comment explaining the Makefile reads it), and remove the inert args: the pre-`FROM` `R_VERSION` and the duplicate pre-`FROM` `USERNAME`, plus the post-`FROM` `INSTALL_MODE`, which nothing references either (the install mode is recorded in the `zzcollab.install.mode` `LABEL` and the `ZZCOLLAB_INSTALL_MODE` `ENV` that the runtime actually reads). Each removal was checked against the Makefile and other tooling, not just the build. A cleaner long-term design would have the Makefile recover the base image from the `FROM` line or the `zzcollab.base.image` `LABEL` rather than from an `ARG`, after which `BASE_IMAGE` could also be dropped; that Makefile change is deferred. Lesson recorded: 'the build does not reference it' does not establish that a line is dead when out-of-band tooling parses the generated file.

### F-2. Runtime renv library is not writable by the run user

The `renv` install block creates `/opt/renv/library` and `/opt/renv/cache` as root with `chmod 755`. The only ownership change in the file is `chown -R ${USERNAME}:${USERNAME} /usr/local/lib/R/site-library`; `/opt/renv`

is left root-owned. The container runs as the non-root `analyst`, with `RENV_PATHS_LIBRARY=/opt/renv/library`. With `ZZCOLLAB_AUTO_RESTORE=false` and a baked library, the read path is intact, so routine use is unaffected. However, any runtime package installation into the renv library, the 'Layer 2, per-user packages added at runtime' half of zzcollab's stated two-layer model, will fail with a permission error because `analyst` cannot write under root-owned `/opt/renv`. If Layer 2 is a supported workflow for this profile, the file contradicts it. This must be reconciled against the intended runtime model before a fix is chosen: either `chown` the renv tree to the user, or document that Layer 2 installs target a different, user-writable library path. Profile-independent.

### F-3. ABI compatibility rests on a hardcoded codename heuristic

The PPM URL pins binaries for Ubuntu `noble`. Those binaries are ABI-compatible only if the rocker base is itself noble-based. The codename is not probed from the base image; it is inferred from the R version by the `case` in `get_ubuntu_codename`, whose default arm returns `noble` for any unrecognised version. This is correct for current rocker images but is a standing assumption that will drift silently when rocker changes its base OS for a future R release: the generator would continue to emit `noble` while the base moved on, yielding ABI-mismatched binaries, silent source fallback, or load failures. A more robust design reads the codename from the resolved base image (for example via `/etc/os-release`) rather than mapping from the R version. Profile-independent.

### F-4. USERNAME is declared twice

`ARG USERNAME=analyst` is emitted before `FROM` and again after it. Build arguments do not survive a `FROM`, so the pre-`FROM` declaration is unused in this single-stage file. It is harmless but should be removed for clarity, unless retained deliberately to support a future multi-stage rewrite, in which case a comment should say so.

### F-5. zzrenvcheck is named but not installed in the image

The generator emits a comment stating that zzrenvcheck is installed post-build via `make install-zzrenvcheck`, and computes a `zzrenvcheck_tag`/`zzrenvcheck_version`, but no `RUN` actually installs it. The project's `tooling.lock` pins `rgt47/zzrenvcheck@v0.3.1`, which the image therefore does not contain. The deferral is defended in the comment by GitHub and cloud-mounted-filesystem issues during build. The consequence is that the image alone cannot run dependency validation; the reproducibility envelope depends on a post-build step outside the Dockerfile. This is a defensible trade-off but should be stated as an explicit limitation of the image's self-sufficiency, not left implicit.

### F-7. renv is installed into the system library only to bootstrap the restore

The renv install block runs `install.packages('renv')` into the system library, then `renv::init(bare=TRUE); renv::restore()`. Because the renv library `/opt/renv/library` is empty until `restore()` populates it, renv must be loadable beforehand, and the system-library install is that bootstrap: without it the `renv::init`/`renv::restore` line fails with 'no package called renv'. The need is therefore real at build time. At runtime it is redundant: renv is also a lockfile entry, so `restore()` installs the pinned renv into `/opt/renv/library`, which is first on `.libPaths()`, and the system-library copy is never loaded. Two minor liabilities follow. First, the bootstrap install is unpinned: it takes whatever the dated PPM snapshot serves, which need not match the lockfile pin. This is no longer hypothetical. A PPM query for the specimen's snapshot (Section 8) shows the latest renv at `noble/2026-06-27` is **1.2.3**, whereas `renv.lock` pins **1.1.5**. The build therefore bootstraps `restore()` with renv 1.2.3 in the system library and then installs renv 1.1.5 into the renv library, so the renv that performs the restore is not the renv the image runs. The two versions are close and the restore succeeds, but the skew is a real divergence between the build tool and the recorded environment. Second, the system-library install is a separate, cache-coarse layer that bakes a package the running image never loads.

Status: resolved by F-9. The version-skew window described here is what F-9 later measured as a concrete cost (the pinned renv compiles from source) and fixed with `renv::restore(exclude = 'renv')`. An alternative considered here, replacing the bootstrap with renv's own `renv/activate.R` to install the pinned version

directly, was rejected: it is more invasive (it requires copying the `renv/` infrastructure into the build) and F-9's exclude is simpler and sufficient. Profile-independent.

Design note, recorded here because the generator comments do not state it: the system library holds unpinned tooling (`languageserver`, `yaml`) while the renv library holds version-pinned project dependencies (`tinytest`, renv itself), with `renv.lock` authoritative and first on the runtime library path. renv is the one package that legitimately appears in both locations, because it must bootstrap its own restore; F-7 concerns whether that system-library appearance can be removed, not whether renv belongs in the lockfile (it does).

#### F-8. The tools install is serial; no Ncpus parallelism

`generate_tools_install` emits `RUN R -e "install.packages(c('languageserver', 'yaml'))"` with no `Ncpus` argument and no `options(Ncpus=)` in scope, so R installs the ~40-package `languageserver` closure one at a time. The verbose build (Section 8.5) showed this was the dominant single cost of the build: all binary downloads complete in ~1.3 seconds, but the serial install phase then runs from t=8 to past t=100 at roughly 1.5 seconds per package. The packages are all binaries with no inter-package build dependency at install time, so the phase parallelises cleanly. Fix (applied): set `Ncpus = max(1L, parallel::detectCores())` on the install call. Verified: the languageserver install fell 103.2 -> 35.8 seconds. Because `languageserver` is a fixed requirement of the vim / zzvim-r workflow, parallelising its install is the principal way to reduce its cost. Profile-independent.

#### F-9. renv::restore compiles the pinned renv from source (fixed)

In the verbose build, `renv::restore()` installed `tinytest` as a binary but compiled `renv` 1.1.5 from source (`renv 1.1.5 [built from source in 1.2m]`), which was almost the entire 85-second restore layer. The cause, established by a direct PPM query, is that the minimal lockfile pins renv 1.1.5, a version PPM no longer serves as a binary at the dated snapshot: a request for `renv_1.1.5.tar.gz` at `noble/2026-06-28` returns HTTP 404 (the version is archived), while the current renv 1.2.3 returns a binary. PPM keeps only a package's current version as a binary in a dated snapshot; older pinned versions are archive-only and therefore source. tinytest 1.4.3, being current, got a binary in the same restore. The differentiator is version currency.

An initial hypothesis, that renv compiles itself from source when restoring its own package while loaded, was tested and withdrawn; so were two download-path remedies (installing `curl`, and `RENV_DOWNLOAD_METHOD=libcurl`), neither of which a missing binary could be cured by. The full sequence is in Section 8.7.

Fix (applied): `renv::restore(exclude = 'renv')`. Excluding renv from the restore skips the archive-only reinstall and leaves the binary bootstrap from the preceding `install.packages('renv')` layer in place. Verified: renv restore 85.3 -> 13.5 seconds; a runtime check loads renv 1.2.3 from the system library and tinytest 1.4.3 from the project library with no network, so the image is offline-correct. The lockfile's hardcoded renv version remains a latent staleness, tracked separately (Section 5, open). This finding also qualifies Section 8.2: PPM having a renv binary did not mean a binary was installed for this package. Profile-independent.

## 4. Confirmed strengths

To avoid optimization regressing what already works, the following are recorded as deliberate and correct:

- Digest-pinned `FROM` for a content-addressed base.
- Dated PPM snapshot pinned in three coordinated locations.
- `RENV_LOCK_HASH` cache-busting of the restore layer.
- Install mode recorded as both `ARG` and `LABEL`.
- renv library sited outside the project bind-mount so it is not shadowed at runtime, with `ZZCOLLAB_AUTO_RESTORE=false` making the baked library authoritative.

## 5. Optimization opportunities

This section is reconciled with the verified outcomes of Sections 8.5-8.7 and the change log (Section 6). Items are grouped by status: **done** (acted on and verified), **rejected** (tried or analysed and found not worthwhile),

and **open** (still candidate, not yet acted on).

**Constraint (fixed).** `languageserver` must remain installed: it is required by the vim / zzvim-r editing workflow this profile targets. It was the single largest cold-build line item (103.2s), but it is a hard dependency, not slack. It is addressed by parallelising its install (F-8, done), not by removal or caching.

**Done (see Section 6)**

- **F-8, parallel tools install.** `Ncpus = max(1L, parallel::detectCores())` on the `languageserver/yaml` install. Verified: 103.2 -> 35.8 seconds.
- **F-9, renv restore as binary.** `renv::restore(exclude = 'renv')`, after finding the lockfile pins a renv version PPM no longer serves as a binary. Verified: 85.3 -> 13.5 seconds, runtime-correct.

**Rejected (tried, not worthwhile)**

- **BuildKit cache mounts (apt and renv).** Implemented per the official patterns, then removed. A valid warm test (Section 8.7) showed the apt mount skips the download but yields no net speedup, because apt is bounded by `apt-get update` and package unpack; and once F-9 removed the renv source build, the renv mount had nothing costly to cache. The earlier `--no-cache` warm test was invalid (it empties mounts). Cache mounts also do not help fresh CI runners, the reproducibility-critical path. Net: added complexity for no measured benefit. (Supersedes the gap noted in `docs/zzcollab-system-review.md`; the gap is real but closing it does not pay off for this workload.)
- **F-7 pinned-renv `activate.R` bootstrap, and remedies A/B.** The renv source build is fixed more simply by F-9 (exclude renv from the restore), so the `activate.R` bootstrap is unnecessary. Installing `curl` (A) and forcing `RENV_DOWNLOAD_METHOD=libcurl` (B) were tried and verified not to affect the outcome, since the cause was binary availability, not the downloader.

**Open (candidate, not yet acted on)**

- **F-1, inert-arg removal.** Drop the unused `ARG BASE_IMAGE` / `ARG R_VERSION`, or wire them into `FROM`.
- **F-3, codename derivation from the base image.** Read the Ubuntu codename from the resolved base image rather than mapping it from the R version, so PPM binary requests cannot silently mismatch the base OS.
- **F-2, renv library ownership.** Resolve the non-root write path to the renv library if runtime ('Layer 2') installs are to be supported.
- **F-5, zzrenvcheck self-sufficiency.** Decide whether to document the post-build install as an explicit image limitation.
- **Stale hardcoded renv pin.** Independent of F-9's restore fix, the minimal lockfile hardcodes renv 1.1.5 (`create_renv_lock_minimal`); a maintained or derived current version would also avoid the staleness at its source.
- **Layer consolidation.** Low priority; needs measurement, since merging `RUN` layers trades rebuild granularity for fewer layers.

## 6. Change log

Each entry records the finding addressed, the generator edit, and the verification performed (build result, image labels, and any runtime check).

**CL-1. Build-optimization batch (branch `feat/dockerfile-build-optimization`)**

Applied to `modules/docker.sh` and build-tested with a clean `zzc rebuild --no-cache --log` (Section 8.6). Outcomes verified.

- **F-8 / Edit 4 (Ncpus). Verified effective.** `generate_tools_install` now calls `install.packages(c('languageserver', Ncpus = max(1L, parallel::detectCores()))`. The `languageserver` install fell 103.2 -> 38.0 seconds.

- **F-9 remedy A (curl). Verified ineffective for F-9.** `curl` is installed unconditionally in the tools layer; the image now has the CLI (`Sys.which('curl')` non-empty), but renv still built itself from source. curl retains independent utility but did not achieve its stated purpose.
- **F-9 remedy B (download method). Verified ineffective for F-9.** `RENV_DOWNLOAD_METHOD=libcurl` was added; renv still built from source (95.7s). At this point the cause was wrongly attributed to renv self-installing while loaded; CL-2 later established it was the stale lockfile pin (renv 1.1.5 is archive-only at the snapshot) and applied the real fix, `renv::restore(exclude='renv')`. Remedies A and B were reverted.
- **Edit 1 (renv cache mount). Applied; correctness verified.** The `renv::restore()` layer gains `--mount=type=cache,target=/opt/renv/cache,sharing=locked`. Warm-rebuild speedup not yet measured.
- **Edit 2 (symlink safeguard, mandatory for Edit 1). Verified correct.** `RENV_CONFIG_CACHE_SYMLINKS=FALSE` added to the ENV block; the runtime library holds real copies and both packages load (Section 8.6).
- **Edit 3 (apt cache mount). Applied.** The tools apt layer gains `/var/cache/apt` and `/var/lib/apt/lists` cache mounts, with `docker-clean` removed and `Keep-Downloaded-Packages` enabled. Warm- rebuild speedup not yet measured.

Remaining for this batch: the real F-9 fix (exclude renv from restore), a warm-rebuild measurement of the cache mounts, and a decision on whether to keep curl / `RENV_DOWNLOAD_METHOD`. Now completed: the runtime `library(tinytest)` / `library(renv)` load check as the baked-library user, and the verbose-log inspection of the renv layer, which confirmed renv installs from source (not a binary) under remedies A and B. Net measured effect of CL-1: total build 233.1 -> 174.2 seconds (-25%), all from F-8; the F-9 source build is unfixed and now the dominant layer.

### CL-2. Real F-9 fix and simplification (supersedes parts of CL-1)

Subsequent investigation (Section 8.7) found the true F-9 cause and a valid cache-mount test, and the branch was simplified to two changes.

- **F-9 real fix: `renv::restore(exclude = 'renv')`.** The renv source build was caused by the lockfile pinning renv 1.1.5, a version PPM no longer serves as a binary at the snapshot (404; only the current 1.2.3 has a noble binary). Excluding renv from the restore leaves the fast binary bootstrap (`install.packages('renv')`, current version) in place. Verified: renv restore 95.7 -> 13.5 seconds, and a runtime check loads renv 1.2.3 from the system library with no network access. The earlier self-install hypothesis is withdrawn.
- **Cache mounts (Edits 1-3) reverted.** The CL-1 warm-rebuild test was invalid: `docker build --no-cache` empties cache mounts each build (moby/moby #41715), so it never tested a warm cache. A valid warm test (build-arg bust, no `--no-cache`) showed the apt cache mount does skip the download but yields no net speedup, because apt is dominated by `apt-get update` and package unpack, not download. With the F-9 source build eliminated, the renv cache mount also had nothing costly left to cache. The mounts added Dockerfile complexity for no measured benefit and were removed, along with `RENV_CONFIG_CACHE_SYMLINKS` (only needed for the renv mount) and remedies A (`curl`) and B (`RENV_DOWNLOAD_METHOD`), which never affected F-9.
- **Final branch state:** two generator changes only, F-8 (Ncpus) and F-9 (`exclude = 'renv'`). Net cold build 233.1 -> 91.8 seconds (-61%), shellcheck-clean, runtime-verified.

## 7. Verification plan

Inspection findings will be confirmed, and changes validated, by:

- Building the image for the specimen project and recording build success, duration, and final size.
- Inspecting the resulting image labels to confirm provenance fields.
- A runtime check of the renv library write path (F-2) by attempting a Layer 2 install as the `analyst` user.
- For F-3, reading `/etc/os-release` from the resolved base image and comparing to the emitted codename.

Section 6 records, for each change, whether it was confirmed by an actual build, a runtime test, or a registry

query.

## 8. Binary-install verification

This section records the first verification performed against the specimen: whether the packages the image installs arrive as precompiled binaries or are built from source. Build time is dominated by source compilation of packages with native code; the objective is that every such package install as a binary. The check was performed by querying Posit Package Manager directly, not by building the image, and is therefore confirmed by query, not by building the image.

### 8.1 Method

PPM serves Linux binaries from the same `src/contrib` URL as source packages; the response is a binary when two conditions hold: the repository URL carries a `__linux__/<codename>` segment, and the HTTP `User-Agent` identifies R on a Linux platform (carrying, for example, `x86_64-pc-linux-gnu`). The generator satisfies both: the dated `__linux__/noble` URL is written into `Rprofile.site` and `RENV_CONFIG_REPOS_OVERRIDE`, and `Rprofile.site` sets an `HTTPUserAgent` option that embeds `R.version["platform"]`.

The discriminator is observable in the response headers. A `HEAD` for `yaml_2.3.12.tar.gz` at `noble/2026-06-27` with a platform-bearing R User-Agent returns `x-package-type: binary` and `x-package-binary-tag: 4.6-noble` (123,783 bytes); the same URL without the platform User-Agent returns the source tarball (109,080 bytes). Each package the image installs, plus the compiled-heavy transitive dependencies of `languageserver`, was probed this way for its version at the snapshot.

Preconditions confirmed from PPM `__api__/status`: binaries are enabled; R 4.6 is a built binary version (`r_versions: 4.6, 4.5, 4.4, 4.3, 4.2, 3.6`); and `noble` (Ubuntu 24.04) publishes binaries for both `x86_64` and `arm64`. The image is built `--platform linux/amd64`, so the relevant architecture is `x86_64`.

### 8.2 Result

Every probed package is served as a `4.6-noble` binary. There were no source fallbacks.

| Package | Version | Native code | Served as |
|---|---|---|---|
| yaml | 2.3.12 | yes | binary |
| renv | 1.2.3 | no | binary |
| tinytest | 1.4.3 | no | binary |
| languageserver | 0.3.18 | yes | binary |
| stringi | 1.8.7 | yes | binary |
| xml2 | 1.6.0 | yes | binary |
| jsonlite | 2.0.0 | yes | binary |
| fs | 2.1.0 | yes | binary |
| roxygen2 | 8.0.0 | yes | binary |
| collections | 0.3.12 | yes | binary |
| R6 | 2.6.1 | no | binary |
| lintr | 3.3.0-1 | no | binary |
| styler | 1.11.0 | no | binary |
| xmlparsedata | 1.0.5 | no | binary |
| callr | 3.8.0 | no | binary |

The build-time-dominant compiled packages (`stringi`, the largest, ships as a ~3.5 MB binary in place of a multi-minute C++ compilation; also `xml2`, `jsonlite`, `fs`) all resolve to binaries. The system package `pandoc` is installed from the Ubuntu apt repository and is likewise not compiled. No change to the Dockerfile is required to meet the all-binary objective for this specimen.

### 8.3 Scope and conditions

- The result was confirmed by querying PPM and reading its response headers, not by building the image. For the `install.packages()` calls this is conclusive, because R fetches from the same URLs with the same `HTTPUserAgent`. The `renv::restore()` packages (`renv`, `tinytest`) are downloaded by renv's own path; both have no native code, so a source fallback there would cost no compilation time and would not affect the objective.
- The probe covered every direct install and the compiled-heavy dependencies of `languageserver`. The remaining pure-R transitive dependencies were not individually probed; lacking native code, they carry no compilation cost regardless of binary status.
- The result is specific to `noble`, R 4.6, and the `2026-06-27` snapshot. PPM builds binaries asynchronously after a source version is published, so a snapshot taken immediately after a new R release or a fresh package version can transiently lack a binary. This snapshot is complete.

### 8.4 What would break the all-binary property

The binary path depends on the emitted Ubuntu codename matching the base image's actual OS. This couples Section 8 to finding F-3: if a future R release's rocker base moved off `noble` while `get_ubuntu_codename` continued to emit `noble`, the image would request `noble`-built binaries for an OS that is not noble. F-3 is therefore not only an ABI-correctness concern but a binary-reliability one, and deriving the codename from the resolved base image rather than from the R version would protect both.

The probe also produced the concrete version evidence cited in F-7: the latest renv at this snapshot is 1.2.3 while the lockfile pins 1.1.5, confirming the bootstrap-versus-restore skew.

### 8.5 Cold-build timing

A cold build of the specimen was run (`zzc docker`, snapshot regenerated to `2026-06-28`). Only the `FROM` layer was cached; every `RUN` executed fresh. Total wall time was 233.1 seconds, distributed as follows.

| Step | Layer | Time (s) |
|---|---|---|
| 3/10 | `apt-get install pandoc` | 32.2 |
| 4/10 | `install.packages(languageserver, yaml)` | 103.2 |
| 5/10 | `install.packages(renv)` | 8.1 |
| 8/10 | `renv::init(bare) + renv::restore()` | 85.3 |
| other | FROM (cached), copy, useradd, export | ~4 |

A verbose no-cache rebuild (`zzcollab rebuild --no-cache --log`, `test.log`) captured per-package install detail and resolves both hot spots to verified causes. It also confirms Section 8.2 at build level: the `languageserver` step logs `* installing *binary* package` for every one of its ~40 dependencies (`stringi`, `xml2`, `jsonlite`, `fs`, `roxygen2`, and the rest), with no `*source*` install anywhere. The Section 8.2 result, first confirmed by query, is therefore now confirmed by an actual build.

First, the `languageserver` 103 seconds is serial per-package install overhead, not download and not compilation. The verbose log shows all ~40 binary downloads completing in roughly 1.3 seconds (t=4.9 to t=6.2), after which packages install strictly one at a time, each separated by about 1.5 seconds (`ps` at t=8.2, `evaluate` at t=9.8, and so on to `R.utils` at t=61). The unpack of each binary is about 0.09 seconds; the ~1.5-second gap is per-package install machinery, applied serially because `install.packages(c('languageserver','yaml'))` is called with no `Ncpus`. This is finding F-8.

Second, the 85.3-second `renv::restore()` is no longer unexplained: renv 1.1.5 was built from source. The log records `renv 1.1.5 [built from source in 1.2m]` alongside `tinytest 1.4.3 [installed binary]` in the same restore, with `Successfully installed 2 packages in 73 seconds` and a `curl does not appear to be installed; downloads will fail` warning, and an earlier `Querying repositories for available source packages` line. So the restore obtained a binary for tinytest but fell to the source path for renv and spent 73

seconds building it. This is a different defect from F-8 and is recorded as finding F-9; neither `Ncpus` nor a cache mount on a cold build addresses it.

No layer in this build used a BuildKit cache mount: the specimen declares no R-package system requirements, so the one cache-mounted path the generator does emit (the derived-system-deps apt block) was not produced, and the pandoc and renv layers have no mount in any path. Each of these costs is therefore paid in full on every cold build. This is the empirical basis for the cache-mount item in Section 5.

### 8.6 Post-change cold build

The branch `feat/dockerfile-build-optimization` was regenerated and built clean (`zzc rebuild --no-cache --log`) on the same host and snapshot. Total fell from 233.1 to 174.2 seconds (-59s, -25%), entirely from F-8.

| Layer | Baseline | Branch | Delta |
|---|---|---|---|
| apt (pandoc + curl) | 32.2 | 29.7 | -2.5 |
| `install.packages(languageserver…)` | 103.2 | 38.0 | -65.2 |
| `install.packages('renv')` bootstrap | 8.1 | 8.2 | +0.1 |
| `renv::restore()` | 85.3 | 95.7 | +10.4 |
| other | ~4 | ~2.4 | - |
| **total** | 233.1 | 174.2 | -58.9 |

Verified conclusions:

- **F-8 (Ncpus) works.** The `languageserver` install fell 103.2 -> 38.0 seconds (-63%); the log shows R's parallel-install path (`begin installing package …`) rather than the serial sequence.
- **F-9 remedies A and B do not work** (finding F-9). The renv source build persisted at 95.7 seconds, marginally slower than baseline (source-build variance), confirming neither curl nor the libcurl download method changes the outcome, because the cause is binary availability (the pinned renv is archive-only at the snapshot), not the downloader. This layer was the dominant cost until the exclude-renv fix in Section 8.7.
- **Edits 1 and 2 are correct.** A runtime check (`R --vanilla`, library path set to the renv platform tree) loaded both `renv` 1.1.5 and `tinytest` 1.4.3 from the baked library, and the library entries are real directories, not symlinks into the discarded cache mount. So `RENV_CONFIG_CACHE_SYMLINKS=FALSE` made the library self-contained as intended. (An initial check failed only because it pointed at the library root rather than the nested `linux-ubuntu-noble/R-4.6/x86_64-pc-linux-gnu` path; the image was never broken.)

Not yet measured: the warm-rebuild benefit of the cache mounts (a second build reusing the apt and renv caches), which is the change that the cache mounts target and which this cold build cannot show.

### 8.7 Final result, valid warm test, and the real F-9 fix

After CL-1, three things were established and the branch was reduced to its final form.

**The real F-9 cause, confirmed.** A direct PPM query shows renv 1.1.5, the version the generator hard-codes into the minimal lockfile, returns HTTP 404 at the noble/2026-06-28 snapshot, while the current renv 1.2.3 returns a binary. PPM keeps only the current version of a package as a binary in a dated snapshot; older pinned versions are archive-only and therefore source. So renv compiled from source because the lockfile pinned a non-current renv, not because renv self-installs. tinytest 1.4.3 (current) got a binary in the same restore. The fix is `renv::restore(exclude = 'renv')`: skip reinstalling renv, and let the `install.packages('renv')` bootstrap (which fetches the current binary) stand. Measured: renv restore 95.7 -> 13.5 seconds. A runtime check loads renv 1.2.3 from the system library and tinytest 1.4.3 from the project library with no network, so the image remains offline-correct.

**The cache-mount test was invalid; a valid one shows no benefit.** The CL-1 warm rebuild used `docker build --no-cache`, which empties cache mounts on every build (moby/moby #41715); it never measured a

warm cache. A valid test (isolated apt layer, build-arg bust, no `--no-cache`) confirmed the mount was warm and that the `.deb` download was skipped, yet the layer was no faster (14.9 vs 17.8 seconds), because apt's cost is `apt-get update` and package unpack, not download. With F-9's source build removed, the renv cache mount also had nothing expensive to cache. The cache mounts were therefore reverted.

**Final cold build, F-8 + F-9 only:**

| Layer | Baseline | Final |
|---|---|---|
| apt (pandoc) | 32.2 | 32.3 |
| `install.packages(languageserver…)` | 103.2 | 35.8 |
| `install.packages('renv')` bootstrap | 8.1 | 8.0 |
| `renv::restore(exclude='renv')` | 85.3 | 13.5 |
| other | ~4 | ~2.2 |
| **total** | 233.1 | 91.8 |

Two generator changes (Ncpus, exclude renv) cut the cold build 61% with no added Dockerfile complexity. The remaining large layers, apt (32s) and the parallelised languageserver install (36s), are both bounded by work the image genuinely needs (installing pandoc, installing the language server) and are not pursued further.

### 8.8 Cross-profile verification

Because F-8 and F-9 are emitted by the shared generator, they apply to all three profiles unchanged. To confirm the benefit transfers rather than assume it, the generated Dockerfile for the `tidyverse` and `rstudio` profiles was built clean (`docker build --no-cache`), using the same minimal renv+tinytest lockfile as the specimen. Both built successfully, both carry `Ncpus` and `renv::restore(exclude = 'renv')`, and both correctly omit the pandoc install (their bases bundle it). Base-image pull is separated from build work because it is a one-time, cache-dependent cost.

| Layer | minimal | tidyverse | rstudio |
|---|---|---|---|
| pandoc install | 32.3 | (in base) | (in base) |
| languageserver install (F-8) | 35.8 | 18.8 | 36.7 |
| languageserver packages installed | ~43 | 23 | full closure |
| renv bootstrap | 8.0 | 7.8 | 8.3 |
| renv restore (F-9) | 13.5 | 13.3 | 13.7 |
| renv built from source? | no | no | no |
| useradd + chown | 1.8 | 8.7 | 1.9 |
| **build work subtotal** | **91.8** | **49.7** | **61.0** |
| base-image pull | cached | 20.3 | cached* |

* rstudio's base was already cached (it shares layers with tidyverse); a cold machine pulls ~2 GB like tidyverse.

Verified conclusions:

- **F-9 is profile-invariant.** renv restore is 13.5 / 13.3 / 13.7 seconds across the three profiles, with no renv source build in any; the ~70-second fix transfers identically, as expected from its shared origin in `create_renv_lock_minimal`.
- **F-8 transfers, with magnitude set by base overlap.** tidyverse installs only 23 of the language-server closure because its baked tidyverse stack already supplies much of it (rlang, cli, jsonlite, stringi, xml2); rstudio, whose base lacks that stack, installs the full closure and matches minimal.
- **The heavier profiles do less build work** (49.7 / 61.0 vs 91.8 seconds), because they skip the pandoc install minimal must perform, and tidyverse's overlap shrinks the language-server install. The cost shifts to the larger base-image pull, not the build.

Scope limit: the test used the minimal lockfile for all profiles, not a realistic tidyverse or rstudio project lockfile. A larger real lockfile that pins archived package versions would source-compile those packages by the same mechanism as renv 1.1.5, independent of profile and of this fix.

## 9. Anatomy of the generated Dockerfile

This section explains every instruction the generator emits, why it is present, and how it changes between profiles. It is written for a reader fluent in R and data analysis but not necessarily in Docker; terms of art are defined in Appendix C. Line numbers refer to the final specimen in Appendix A.

### 9.1 Header and base image

- `# syntax=docker/dockerfile:1.4` (line 1). Selects the BuildKit Dockerfile frontend. It must be the first line; it enables newer features and is required for the `RUN --mount` syntax the generator uses elsewhere.
- `# zzcollab Dockerfile vX` (line 2). A provenance comment recording the template version that produced the file.
- `ARG BASE_IMAGE` (lines 4-6). The `FROM` line is a fully-substituted literal and does not reference it, but the project Makefile parses it back out of the file to label the profile in `make r`, so it is kept (finding F-1). The previously emitted `ARG R_VERSION` and a duplicate pre-`FROM ARG USERNAME` were genuinely unread and were removed; the non-root account name is declared once, after `FROM` (Section 9.2).
- `FROM <base>:<rver>@sha256:<digest>` (line 8). The base image, pinned to an immutable content digest so the build is reproducible even if the tag is later repointed. **This is the principal line that varies by profile** (Section 9.6).

### 9.2 Labels and build arguments

- `LABEL ...` (lines 13-20). Open Container Initiative (OCI) and zzcollab-specific metadata baked into the image: creation time, license, template version, R version, base image and its digest, the PPM snapshot date, and the dependency-install mode. This makes the image self-describing; tools and humans can read its provenance without the source. The base-image and digest fields vary by profile.
- `ARG USERNAME` / `ARG DEBIAN_FRONTEND` (lines 22-23). Build arguments do not survive a `FROM`, so `USERNAME` is re-declared (`useradd`/`USER` read it); `DEBIAN_FRONTEND=noninteractive` suppresses interactive prompts during apt installs (apt reads it from the build environment). A former `ARG INSTALL_MODE` here was removed as inert (F-1): the install mode is recorded only in the `LABEL` and the runtime `ENV`, which are what consume it. The line-number references below the ENV block are therefore one line lower than shown; they are not re-numbered here.

### 9.3 Environment

- `ENV ...` (lines 29-35). Runtime environment variables, in three groups: locale and timezone (`LANG`, `LC_ALL`, `TZ`) for deterministic sorting, encoding, and timestamps; renv paths (`RENV_PATHS_LIBRARY`, `RENV_PATHS_CACHE`) sited under `/opt` so the baked library is not shadowed by the project bind-mount at runtime, plus `RENV_CONFIG_REPOS_OVERRIDE` pinning the dated PPM mirror; and zzcollab flags (`ZZCOLLAB_CONTAINER`, `ZZCOLLAB_INSTALL_MODE`, `ZZCOLLAB_AUTO_RESTORE=false`) that the project `.Rprofile` reads to decide whether to run the renv workflow at startup. These lines are profile-independent (the PPM codename is the same for all R 4.6 rocker bases).

### 9.4 System and R tooling

- `# No additional system dependencies required` or an apt block (line 37). Emitted by a generation-time branch: if the project's R packages declare system requirements (queried from PPM), an `apt-get install` of those libraries appears here; otherwise only the comment. **Varies with the project's package set** (Section 9.6).
- `RUN echo 'options(repos=...)' ... Rprofile.site` (lines 40-43). Writes two R options into the site profile: the dated PPM repository, and an `HTTPUserAgent` string carrying the platform. Together

with the `__linux__/<codename>` repository URL, the user agent is what makes PPM serve precompiled binaries instead of source (Section 8). This is the single most important performance line in the file.
- `RUN apt-get install ... pandoc` (line 46). Installs pandoc, needed for R Markdown and Quarto rendering. **Emitted only for the minimal profile**; the tidyverse and rstudio bases already bundle pandoc, so the generator omits it for them (Section 9.6).
- `RUN R -e "install.packages(c('languageserver','yaml'), Ncpus=...)"` (line 49). Installs the R language server (editor completion and diagnostics, required by the vim / zzvim-r workflow) and yaml. `Ncpus` parallelises the install (finding F-8). Profile-independent.

### 9.5 Dependency install, validation tooling, and user

- renv install block (lines 56-68). In renv mode: install renv from the current snapshot (binary bootstrap); create the renv library and cache directories; `COPY renv.lock`; declare `ARG RENV_LOCK_HASH` so the restore layer's cache key tracks the lockfile content (re-runs `restore` whenever the lockfile changes, preventing a stale baked library); then `renv::init` plus `renv::restore(exclude = 'renv')`, which installs the locked packages as binaries while skipping the archive-only pinned renv (finding F-9). **In DESCRIPTION mode this whole block is replaced** by a `COPY DESCRIPTION` and a `pak::local_install_deps()` call; the choice depends on whether a `renv.lock` is present, not on the profile.
- zzrenvcheck comment (lines 70-72). A placeholder noting that the dependency-validation package is installed post-build (`make install-zzrenvcheck`), not during the image build, to avoid GitHub and cloud-filesystem issues (finding F-5). No instruction is emitted.
- `RUN useradd ... && chown ... site-library` (lines 76-77). Creates the non-root user and gives it ownership of the system R library. Running as a non-root user is a standard container-security practice.
- `USER` / `WORKDIR` / `CMD` (lines 79-82). Switch to the non-root user, set the working directory to the bind-mount point, and define the default command (`R --quiet`). These are profile-independent: even for the rstudio profile the image default is an R session, and RStudio Server is launched separately via `make rstudio` at run time, not baked as the image command.

### 9.6 What changes from profile to profile

The three profiles are `minimal` (base `rocker/r-ver`), `tidyverse` (base `rocker/tidyverse`), and `rstudio` (base `rocker/rstudio`). The generated Dockerfile is deliberately almost identical across them; only a small, well-localised set of lines differs.

| Element | minimal | tidyverse | rstudio |
|---|---|---|---|
| `FROM` base image (line 8) | `rocker/r-ver` | `rocker/tidyverse` | `rocker/rstudio` |
| Base `@sha256` digest | per-image | per-image | per-image |
| `LABEL zzcollab.base.image` | r-ver | tidyverse | rstudio |
| pandoc apt install (line 46) | **present** | omitted (in base) | omitted (in base) |
| Baked R packages | none beyond base | tidyverse stack | tidyverse stack |
| RStudio Server | absent | present (unused by CMD) | present (via `make rstudio`) |

Everything else, the locale and renv environment, the PPM configuration and user-agent, the language-server install, the renv restore logic, the user and `CMD`, is identical. Two further axes of variation are orthogonal to the profile: the system-dependency apt block (line 37) is driven by the project's package system requirements, and the dependency-install block (lines 56-68) is driven by renv-versus-DESCRIPTION mode. The R version selects the Ubuntu codename in the PPM URL (jammy for 4.2-4.3, noble for 4.4+), which is a function of the base image, not the profile.

The design intent is that a project can change profile, or move between them, with a single regenerated line (`FROM`) and at most one added or removed apt line, keeping the reproducibility-relevant configuration constant.

## 10. The ephemeral-library model and where validation may run

This section records an architectural invariant that constrains how the runtime workflow may be wired. It is not a defect; it is a property that any future change to the `make r` workflow or the dependency-validation targets must respect. It follows from environment choices the generated Dockerfile makes (Sections 4 and 9.3), so it belongs with the rest of the Dockerfile's documentation even though it manifests in the Makefile.

### 10.1 The baked-library, ephemeral-install model

The Dockerfile sites the renv library at `RENV_PATHS_LIBRARY=/opt/renv/library` inside the image and sets `ZZCOLLAB_AUTO_RESTORE=false`, so the baked library is the authoritative source of installed packages. The interactive `make r` container bind-mounts the renv *cache* (`/opt/renv/cache`) from the host, but not the *library*. A package a user installs during an interactive session therefore lands in that container's `/opt/renv/library`, which is ephemeral: the container runs with `--rm`, so the library and any session installs are discarded when it exits. Session installs are transient by design; the image library changes only on a rebuild.

### 10.2 Two automatic operations on session exit, in two places

Leaving an interactive `make r` session triggers two steps, by two different mechanisms and in two different locations:

- **`renv::snapshot()` runs inside the container, via R's `.Last` hook.** The project `.Rprofile` defines `.Last`, which R calls during its own shutdown; it runs `renv::snapshot(prompt = FALSE)` to record installed versions into the bind-mounted `renv.lock`. This happens before the container is torn down.
- **`zzrenvcheck::check_packages()` runs afterward, as the next Makefile recipe line.** The interactive `docker run ... R` blocks until R exits; the container (whose only process is R) then stops, control returns to the host `make` recipe, and the recipe's next command runs `check_packages()`. There is no in-container step between R exiting and the container stopping; the two are the same event.

### 10.3 The governing principle: reads-the-library vs reads-the-files

The two steps cannot be wired interchangeably, because they read different things:

- `renv::snapshot()` reads the **installed library** to record exact versions. That state exists only in the live session, so snapshot must run **in that session, at exit** (`.Last`). A fresh container started after the session begins from the baked image library and the bind-mounted cache; it does not contain the session's installs, so a snapshot taken there would silently omit them.
- `zzrenvcheck::check_packages()` reads **files** (`renv.lock`, `DESCRIPTION`, source code), all bind-mounted. It does not need the session library, so it can run in any container that mounts the project, via `$(DOCKER_RUN)` (`docker run --rm -v project ... <image> Rscript -e ...`).

| Step | Reads | Must run | Mechanism |
|---|---|---|---|
| `renv::snapshot()` | installed library (live session state) | in the session, at exit | `.Last` |
| `check_packages()` | project files | any container mounting the project | `$(DOCKER_RUN)` |

### 10.4 Consequences for future modifications

- **Do not move `renv::snapshot()` to a post-session `$(DOCKER_RUN)` call.** A fresh container has the baked library, not the session's installs, so the snapshot would omit newly installed packages and

break the intended loop: install in session, `.Last` snapshots the version into `renv.lock`, the next image build's `renv::restore()` bakes it in. The only way to make a fresh-container snapshot correct would be to bind-mount the library as well, which trades away the baked-library reproducibility model for no benefit, since `.Last` already captures the state at the one moment it exists.

- **Do move the host-side `check_packages()` to `$(DOCKER_RUN)`.** It reads files, needs no session library, and running it on the host requires R on the host. That requirement breaks zzcollab's host-independence and, when no host R is present, the post-session validation silently does not run at all (the most common case for zzcollab users). The on-demand `make check-renv` targets already use `$(DOCKER_RUN)` correctly; only the `make r` post-session call runs on the host. This change is tracked in `docs/zzrenvcheck-validation-plan.md`.
- **Do not move `check_packages()` (or other validations) into `.Last`,** even though that would also run it in-container and solve the host-independence problem. `.Last` runs at the end of *every* container R process, not just interactive sessions: the project `.Rprofile` is sourced for `Rscript` too, so `.Last` would fire after `make render`, `make test`, `R CMD build`, and even the validation call itself. With `auto_fix = TRUE` that would silently edit `DESCRIPTION`/`renv.lock` after routine automated tasks, run redundantly, and couple a deliberate lint to R's shutdown path. `.Last` is the right home for snapshot only because snapshot *must* run in-session (a necessity, not a convenience); a file-reading validation has no such constraint, so it belongs in the make recipe, where its scope (after an interactive `make r`) and location (a fresh `$(DOCKER_RUN)` container) are both correct.
- **General rule for new validation or maintenance steps:** a step that reads the *installed library* must run in the live session (`.Last`); a step that reads *project files* should run via `$(DOCKER_RUN)` at an explicit point in the make recipe, not in `.Last`. Choosing the wrong location yields a step that either misses session state, fires on every R exit, or silently no-ops off-container.

## Appendix A: the generated specimen (peng1 Dockerfile, final optimized form)

This is the Dockerfile after the F-8 and F-9 changes (Section 9 walks through it line by line). It differs from the 233-second baseline only at the `languageserver` install (now `Ncpus`-parallel) and the renv restore (now `exclude = 'renv'`); all other lines are unchanged.

```
# syntax=docker/dockerfile:1.4
# zzcollab Dockerfile v0.1.0

# BASE_IMAGE is read by the project Makefile (make r); the FROM below is a
# fully-substituted literal and does not reference it.
ARG BASE_IMAGE=rocker/r-ver

FROM rocker/r-ver:4.6.0@sha256:6f05a1a8b8c52328f181593923909b01cbfd14c9caea93bf75ddc65e806d8eac

LABEL org.opencontainers.image.created="2026-06-28T02:13:26Z" \
      org.opencontainers.image.licenses="GPL-3.0-or-later" \
      zzcollab.template.version="0.1.0" \
      zzcollab.r.version="4.6.0" \
      zzcollab.base.image="rocker/r-ver:4.6.0" \
      zzcollab.base.digest="sha256:6f05a1a8b8c52328f181593923909b01cbfd14c9caea93bf75ddc65e806d8eac" \
      zzcollab.ppm.snapshot="2026-06-27" \
      zzcollab.install.mode="renv"

ARG USERNAME=analyst
ARG DEBIAN_FRONTEND=noninteractive

ENV LANG=en_US.UTF-8 LC_ALL=en_US.UTF-8 TZ=UTC \
    RENV_PATHS_LIBRARY=/opt/renv/library \
    RENV_PATHS_CACHE=/opt/renv/cache \
    RENV_CONFIG_REPOS_OVERRIDE="https://packagemanager.posit.co/cran/__linux__/noble/2026-06-27" \
```

```
    ZZCOLLAB_CONTAINER=true \
    ZZCOLLAB_INSTALL_MODE=renv \
    ZZCOLLAB_AUTO_RESTORE=false

# No additional system dependencies required

RUN echo 'options(repos = c(CRAN = "https://packagemanager.posit.co/cran/__linux__/noble/2026-06-27"))' \
        >> /usr/local/lib/R/etc/Rprofile.site && \
    echo 'options(HTTPUserAgent = sprintf("R/%s R (%s)", getRversion(), paste(getRversion(), R.version["platform"]
        >> /usr/local/lib/R/etc/Rprofile.site

RUN apt-get update && apt-get install -y --no-install-recommends pandoc && rm -rf /var/lib/apt/lists/*

RUN R -e "install.packages(c('languageserver', 'yaml'), Ncpus = max(1L, parallel::detectCores()))"

RUN R -e "install.packages('renv')"
RUN mkdir -p /opt/renv/library /opt/renv/cache && chmod 755 /opt/renv/library /opt/renv/cache
COPY renv.lock renv.lock
ARG RENV_LOCK_HASH=unknown
RUN echo "renv.lock hash: ${RENV_LOCK_HASH}" && \
    R -e "renv::init(bare=TRUE, force=TRUE, restart=FALSE); renv::restore(exclude = 'renv')"

RUN useradd --create-home --shell /bin/bash ${USERNAME} && \
    chown -R ${USERNAME}:${USERNAME} /usr/local/lib/R/site-library

USER ${USERNAME}
WORKDIR /home/${USERNAME}/project

CMD ["R", "--quiet"]
```

## Appendix B: generator excerpt

The emission is in `modules/docker.sh`, function `generate_dockerfile_inline`, with the codename helper `get_ubuntu_codename`. The `FROM` line is built from the `from_spec` variable (a fully-substituted `base:rver@digest` string), independently of the `ARG BASE_IMAGE` / `ARG R_VERSION` literals the here-document also emits, which is the basis of finding F-1.

## Appendix C: Glossary

Definitions of the terms of art used above, for a reader fluent in R and statistics but not necessarily in container tooling.

### Containers and Docker

- **Docker image**: a read-only, layered filesystem snapshot bundling an operating system, R, packages, and configuration. Analogous to a frozen, shippable computer.
- **Container**: a running instance of an image. The same image run on any machine yields the same environment, which is the reproducibility guarantee.
- **Dockerfile**: the text recipe of instructions (`FROM`, `RUN`, …) from which an image is built. zzcollab generates this file rather than shipping a fixed one.
- **Base image**: the starting image a Dockerfile builds on (`FROM`). Here, a `rocker` image providing a fixed R version on Ubuntu.
- **rocker**: the community project publishing the standard R Docker images (`r-ver`, `tidyverse`, `rstudio`, `verse`).

- **Layer**: each instruction produces one filesystem layer. Images are the stack of these layers; identical layers are shared and cached.
- **Tag** vs **digest**: a tag (`rocker/r-ver:4.6.0`) is a human-readable, mutable pointer; a digest (`@sha256:...`) is an immutable content hash. The generator pins the digest so the base cannot silently change.
- **Bind mount**: a host directory made visible inside the container at run time. The project source is bind-mounted, so edits on the host appear in the container without rebuilding.

**Dockerfile instructions**

- **`FROM`**: declares the base image. **`ARG`**: a build-time variable. **`ENV`**: an environment variable persisted into the image. **`RUN`**: executes a shell command during the build, producing a layer. **`COPY`**: copies a file from the build context into the image. **`LABEL`**: attaches metadata key-values to the image. **`USER`**/**`WORKDIR`**/**`CMD`**: set the default user, directory, and command for containers started from the image.

**Build mechanics and caching**

- **BuildKit**: the modern Docker build engine. The `# syntax=...` line selects its Dockerfile frontend and enables features such as cache mounts.
- **Layer cache**: Docker reuses the cached result of an instruction when the instruction and all preceding ones are unchanged (shown as `CACHED`). A change busts that layer and every layer after it.
- **`--no-cache`**: a build flag that disables the layer cache, forcing every instruction to re-run; it also empties cache mounts (the cause of an invalid test in Section 8.6).
- **Cache mount** (`RUN --mount=type=cache`): a persistent directory, stored outside the image, that survives across builds so a re-run can reuse prior downloads. Investigated and ultimately not retained here (Section 8.7).
- **Content-addressable / hash**: identifying an artifact by a hash of its bytes, so a given hash always denotes identical content. Used for the base digest and the `RENV_LOCK_HASH` cache key.

**Package distribution and compilation**

- **PPM (Posit Package Manager)**: a CRAN mirror that, for supported Linux distributions, serves precompiled package binaries. zzcollab pins a dated PPM **snapshot** (a frozen view of CRAN on a given day) for reproducibility.
- **Binary vs source package**: a binary is precompiled and installs in seconds; a source package must be compiled on the machine, which for packages with C/C++/Fortran can take minutes. **`needs_compilation`** marks packages containing such code.
- **Dependency closure**: a package plus all packages it depends on, transitively. Installing `language-server` pulls a closure of about 40 packages.
- **Archived version**: an older package version PPM keeps only as source in a dated snapshot; a request for its binary returns HTTP 404 (the basis of finding F-9).
- **`HTTPUserAgent`**: an HTTP header identifying the client. PPM uses it, together with the `__linux__/<codename>` URL, to decide whether to serve a Linux binary; this is the mechanism the generator relies on (Section 8).
- **Ubuntu codename** (`noble`, `jammy`): the release name of the Ubuntu version. PPM builds binaries per codename, so it must match the base image OS, or binaries are ABI-incompatible.
- **ABI (Application Binary Interface)**: the low-level compatibility contract between compiled code and the OS/libraries. A binary built for the wrong OS release may fail to load.
- **Byte-compilation**: R's translation of package R code to a compact bytecode at install time; part of why even pure-R source installs are not instantaneous.
- **`Ncpus`**: an `install.packages()` argument setting how many packages install in parallel (default 1, i.e. serial). Raising it parallelises the install (finding F-8).
- **pak**: a modern R package installer used in the generator's DESCRIPTION-install mode.
- **pandoc**: the document converter underlying R Markdown and Quarto rendering; present in the higher rocker images but not `r-ver`.

**renv**

- **renv**: R's project-local dependency manager. It records exact package versions and isolates each project's library.
- **`renv.lock`**: the JSON lockfile listing every package and its exact version; the source of truth for reproducibility (Pillar 2).
- **renv library**: the project-private directory of installed packages (here `/opt/renv/library`). **renv cache**: a shared store of installed packages that libraries link or copy from.
- **`renv::restore()`**: installs exactly the versions recorded in `renv.lock`. **`exclude =`** omits named packages from that restore (the F-9 fix excludes renv itself).
- **`renv::snapshot()`**: the inverse, recording the currently installed versions into `renv.lock`.
- **Bootstrap / `activate.R`**: renv's self-installation. On startup the project `.Rprofile` sources `renv/activate.R`, which makes renv available, downloading it if absent.

**zzcollab terms**

- **Profile**: the Docker base bundle a project builds on (`minimal`, `tidyverse`, `rstudio`); selects the `FROM` image (Section 9.6).
- **Archetype**: the research scaffold and intent (e.g. `analysis`), independent of the Docker profile.
- **System requirements / sysreqs**: the OS libraries an R package needs to build or run (e.g. `libxml2`), installed via `apt`.
- **Five Pillars**: zzcollab's reproducibility set, Dockerfile, `renv.lock`, `.Rprofile`, source code, and data, all version-controlled together.
- **shellcheck**: a static analyzer for shell scripts; the generator's CI requires it to pass.
- **Heredoc**: a shell construct (`<<EOF ... EOF`) for emitting a multi-line block; the generator uses one to write the Dockerfile.

---

*Rendered on 2026-06-28 at 18:56 PDT. Source: ~/prj/sfw/07-zzcollab/zzcollab/docs/dockerfile-template-optimization-whitepaper.md*